

「桌面應用程式開發」 資料庫連結

李東霖 博士

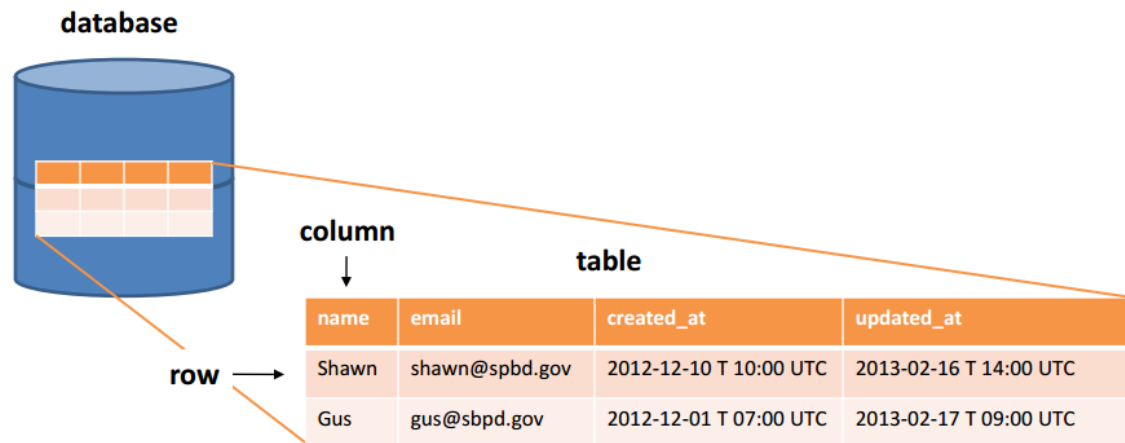
國立臺灣海洋大學 電機工程學系

2025 Fall semester

資料庫簡介

資料庫簡介

- 資料庫（**Database**）是「用來儲存、管理與取用資料的系統」。
- 它像是一個「有組織的資料倉庫」，讓你能快速找到想要的資料。
- 舉例：
 - 學校的學生資料系統：學號、姓名、成績、選課。
 - 網路商店：商品、客戶、訂單、庫存。
 - 手機 **App**（例如 **Instagram**）背後也有資料庫，存放貼文、留言、帳號資訊等。



資料庫 = 書架
資料表（Table）= 一本書
欄位（Column）= 書裡的章節標題
資料列（Row）= 書裡的一頁內容

資料庫和試算表的差異為何？

- 資料庫和試算表 (例如 **Microsoft Excel**) 都是儲存資訊的便捷方式。
兩者之間的主要區別為：
 - 資料的儲存和操控方式
 - 可存取資料的人員
 - 可儲存資料的多寡

項目	試算表（ Excel 、 Google Sheet ）	資料庫（ Database ）
結構	二維表格	多張表格之間可建立關聯
使用方式	手動輸入、人工操作	程式或查詢語言（ SQL ）
適用規模	少量資料	大量資料、多人同時操作
錯誤風險	容易誤刪或格式錯亂	有嚴格的結構與存取控制
效率	慢、難維護	快速查詢、擴充容易

資料庫的類型

1. 關聯式資料庫（Relational Database, RDBMS）

- 資料以「表格（Tables）」形式儲存。
- 各表間可用「鍵值（Keys）」建立關聯。
- 使用 SQL 查詢。
- 例：MySQL、PostgreSQL、Oracle、SQL Server。

2. 非關聯式資料庫（NoSQL）

- 適合非結構化資料（文字、JSON、圖片）。
- 無固定表格結構。
- 類型：
 - 文件型（Document）：MongoDB
 - 鍵值型（Key-Value）：Redis
 - 圖形型（Graph）：Neo4j

3. NewSQL / 雲端資料庫

- 結合關聯式結構與雲端彈性。
- 例：Google BigQuery、AWS Aurora。

關聯式：穩定、有一致性，適合商業系統。

NoSQL：靈活、可擴充，適合社群媒體或 IoT。

全球最熱門資料庫系統

Rank			DBMS	Database Model	Score		
Nov 2025	Oct 2025	Nov 2024			Nov 2025	Oct 2025	Nov 2024
1.	1.	1.	Oracle 企業級、高安全性	Relational, Multi-model ⓘ	1239.78	+27.01	-77.23
2.	2.	2.	MySQL 免費、開源、網站常用	Relational, Multi-model ⓘ	865.82	-13.84	-151.98
3.	3.	3.	Microsoft SQL Server 整合 Windows 生態	Relational, Multi-model ⓘ	718.87	+3.82	-80.94
4.	4.	4.	PostgreSQL 開源、支援複雜查詢	Relational, Multi-model ⓘ	651.36	+8.16	-2.97
5.	5.	5.	MongoDB ⓘ JSON 文件格式、靈活	Multi-model ⓘ	371.68	+3.66	-29.25
6.	6.	↑ 7.	Snowflake 儲存與計算分離、雲端原生架構	Relational	197.84	-0.81	+55.35
7.	7.	↓ 6.	Redis 超快、常用於快取系統	Key-value, Multi-model ⓘ	145.09	+2.76	-3.55
8.	8.	↑ 14.	Databricks	Multi-model ⓘ	131.50	+2.70	+45.04
9.	9.	9.	IBM Db2	Relational, Multi-model ⓘ	119.28	-3.10	-2.47
10.	10.	↓ 8.	Elasticsearch	Multi-model ⓘ	113.97	-2.69	-17.67

<https://db-engines.com/en/ranking>

各種資料庫的適合應用範圍

- 傳統資料庫（ Oracle 、 MySQL 、 PostgreSQL 、 SQL Server ）
 - 適合線上交易、內部系統。
- NoSQL（ MongoDB 、 Redis ）
 - 適合快速存取或非結構化資料。
- 雲端資料庫（ Snowflake ）
 - 專為大數據分析與雲端應用而設計

結構化查詢語言 (SQL)

結構化查詢語言 (SQL)

- SQL (Structured Query Language)是一種專門用來與關聯式資料庫溝通的語言。
- SQL 最早在 1970 年代由 IBM 開發，Oracle 是主要貢獻者，這促成了 SQL ANSI 標準的導入，而 SQL 又帶動了 IBM、Oracle 和 Microsoft 等公司的許多擴展。
- 雖然現今 SQL 仍被廣泛使用，但新的程式設計語言開始出現。
- 主要功能：
 - 定義資料結構（DDL）
 - 例：建立、修改資料表（CREATE TABLE）。
 - 操作資料內容（DML）
 - 例：新增、刪除、修改、查詢（INSERT, DELETE, UPDATE, SELECT）。
 - 控制權限與安全性（DCL）。

SQL基本語法介紹(1)

- 假設已建立一個資料庫(database)
- 資料庫中已建立起一張表(table)名稱為Students
- 表Students中的內容如下:

id	name	department	score
1	Amy	CS	90
2	Bob	EE	85
3	Carol	ME	92

SQL基本語法介紹(2)

- 查詢資料（SELECT）：

`SELECT` name, score `FROM` Students;

→ 顯示所有學生的姓名與成績。

- 新增資料（INSERT）

`INSERT INTO` Students (name, department, score)

`VALUES` ('David', 'CS', 88);

→ 新增一項學生的資料(name='David', department='CS', score=88)

id 一般做為唯一值由系統自動給予編號

SQL基本語法介紹(3)

- 更新資料（UPDATE）

UPDATE Students

SET score = 95

WHERE name = 'Amy';

→ 更新欄位學生name為Amy的score為95。

- 刪除資料（DELETE）

DELETE FROM Students

WHERE id = 2;

→ 刪除欄位id為2的單筆資料

SQL基本語法介紹(4)

- 條件查詢（WHERE + 排序）

```
SELECT * FROM Students  
WHERE score > 85  
ORDER BY score DESC;
```

→以遞減的方式列出score大於85分的學生資料

- 條件查詢（ORDER BY + LIMIT）

```
SELECT name  
FROM Students  
WHERE department = 'CS'  
ORDER BY score DESC  
LIMIT 1;
```

→找出「CS系學生」中分數最高的學生名字

另一解:

```
SELECT name  
FROM Students  
WHERE department = 'CS'  
AND score = (  
    SELECT MAX(score)  
    FROM Students  
    WHERE department = 'CS'  
);
```

連接關聯式資料庫

MariaDB資料庫系統安裝(1)

- MariaDB是MySQL關聯式資料庫管理系統的一個分叉，由社群開發，有商業支援，旨在甲骨文公司收購MySQL後繼續保持在GNU GPL下開源。
- MariaDB的開發是由MySQL的一些原始開發者領導的。與MySQL的高度相容性，與MySQL API和命令精確匹配。又比MySQL有更高的擴展性。
- 維基百科已於2013年正式宣布將從MySQL遷移到MariaDB資料庫
- 官網：<https://mariadb.org/>

MariaDB資料庫系統安裝(2)

User settings

Default instance properties

MariaDB 12.0 (x64) database configuration

☒ **Modify password for database user 'root'**

New root password: Enter new root password

Confirm: Retype the password

☐ **Enable access from remote machines for 'root' user**

☐ **Enable the Feedback plugin and submit anonymous usage information**

Collected statistics can be viewed at <https://mariadb.org/feedback-plugin>

Data directory

Database settings

Default instance properties

MariaDB 12.0 (x64) database configuration

☒ **Install as service**

Service Name:

☒ **Enable networking**

TCP port:

Innodb engine settings

Buffer pool size: MB

Page size: KB

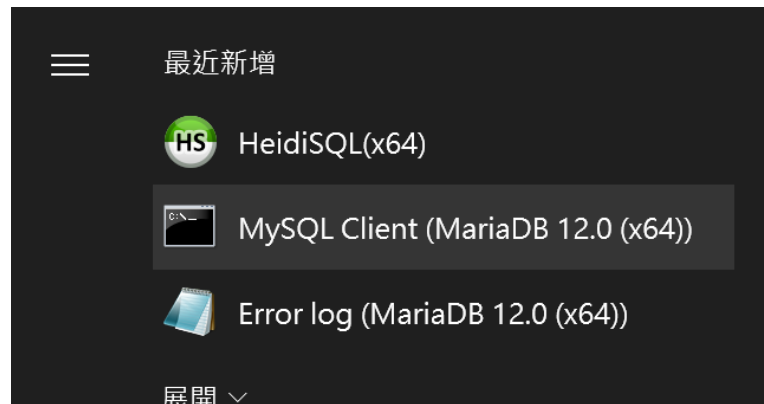
```
Microsoft Windows [版本 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\User>c:

C:\Users\User>netstat -ano | findstr 0.0.0.0:3306
TCP    0.0.0.0:3306          0.0.0.0:0             LISTENING       2968

C:\Users\User>
```


MariaDB資料庫系統安裝(3)



```
MySQL Client (MariaDB 12.0 (x64)) - "C:\Program Files\MariaDB 12.0\bin\mysql.exe" "--defaults-file=C:\Program Files\MariaDB 12.0\data\my.i...
Enter password: *****
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 12.0.2-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

初始化一個資料庫

1. 從MySQL Client端建立資料庫

```
CREATE DATABASE MEMBER;
```

2. 給這個資料庫一個使用者

```
CREATE USER 'T00001'@'localhost' IDENTIFIED BY '123456';
```

3. 重新整理權限

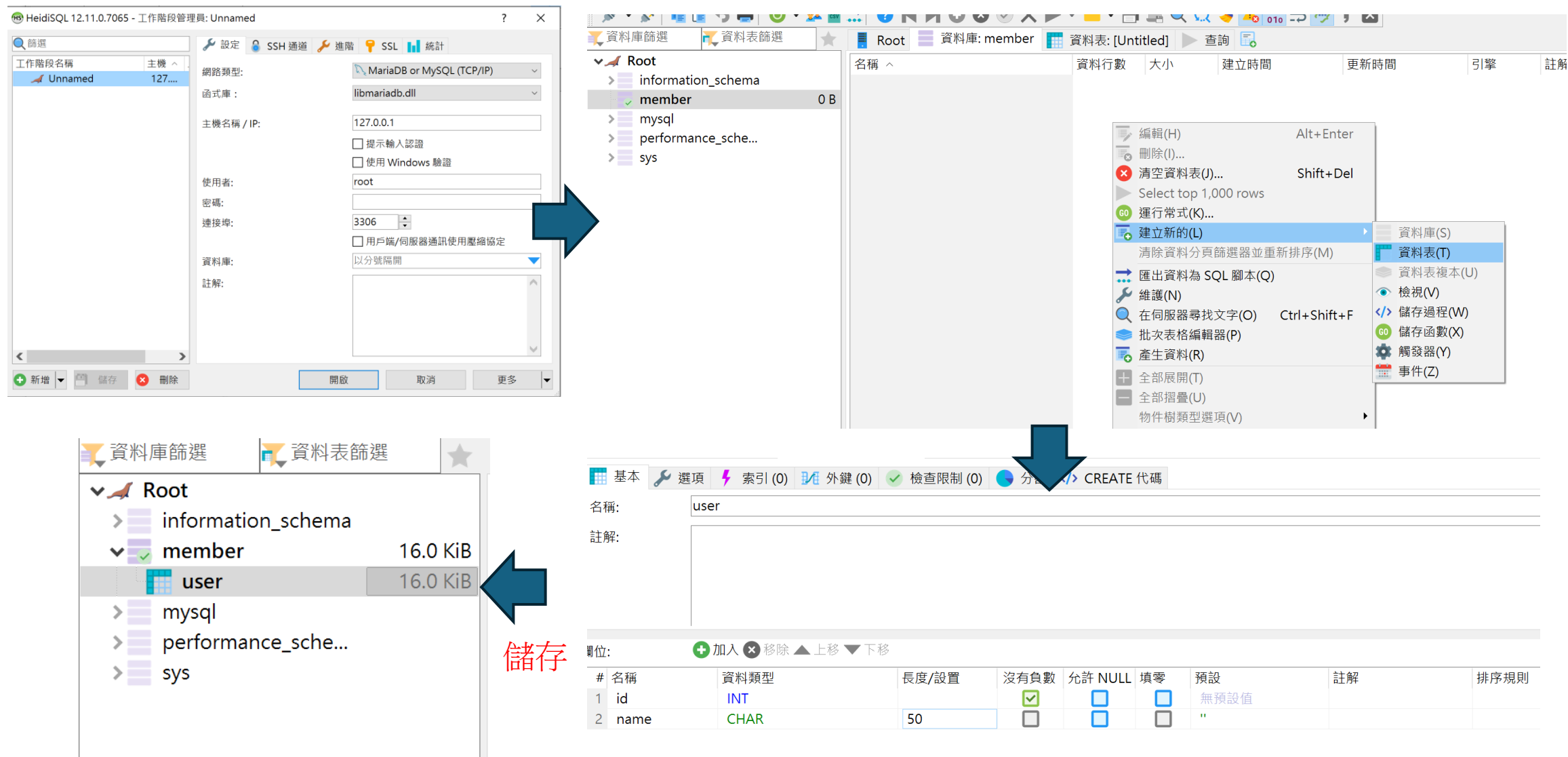
```
FLUSH PRIVILEGES;  
quit -離開資料庫
```

4. 測試登入

```
mysql -u T00001 -p
```

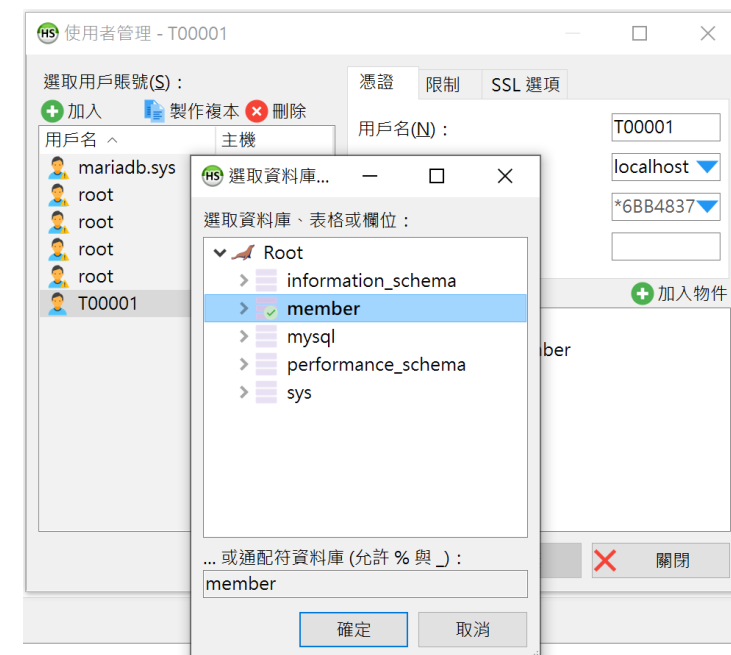
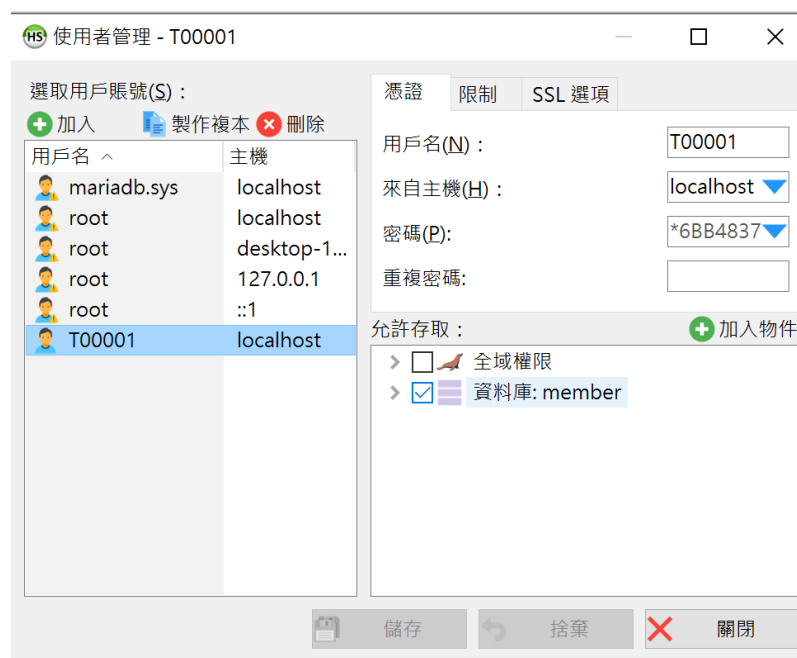
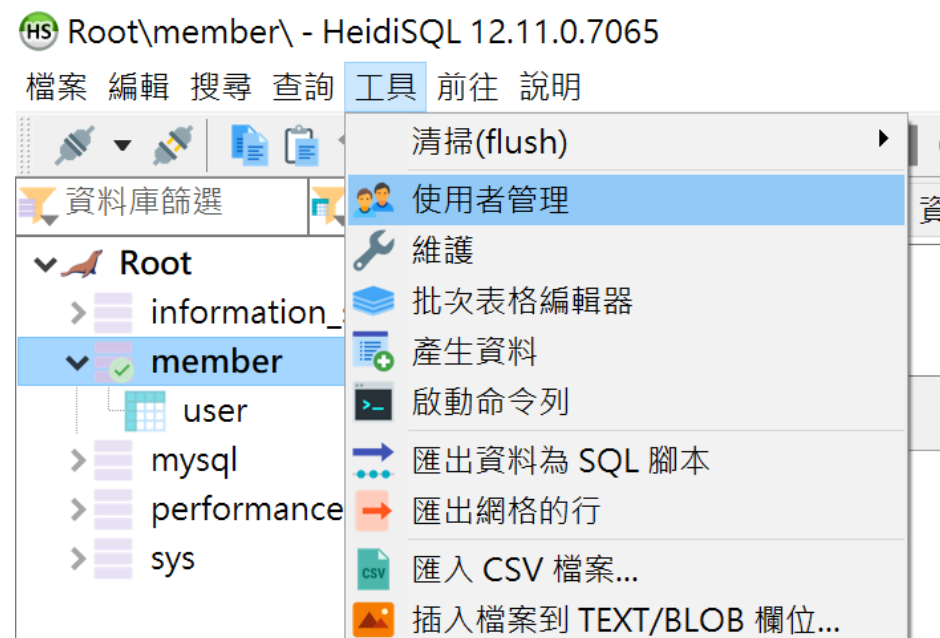
賦予T00001 MEMBER資料庫所有權限並只限於localhost本機IP使用。
你可以將localhost替換成你要開給哪個IP
'123456'是帳號登入密碼。可改成你想要的

透過HeidiSQL在資料庫建立表格

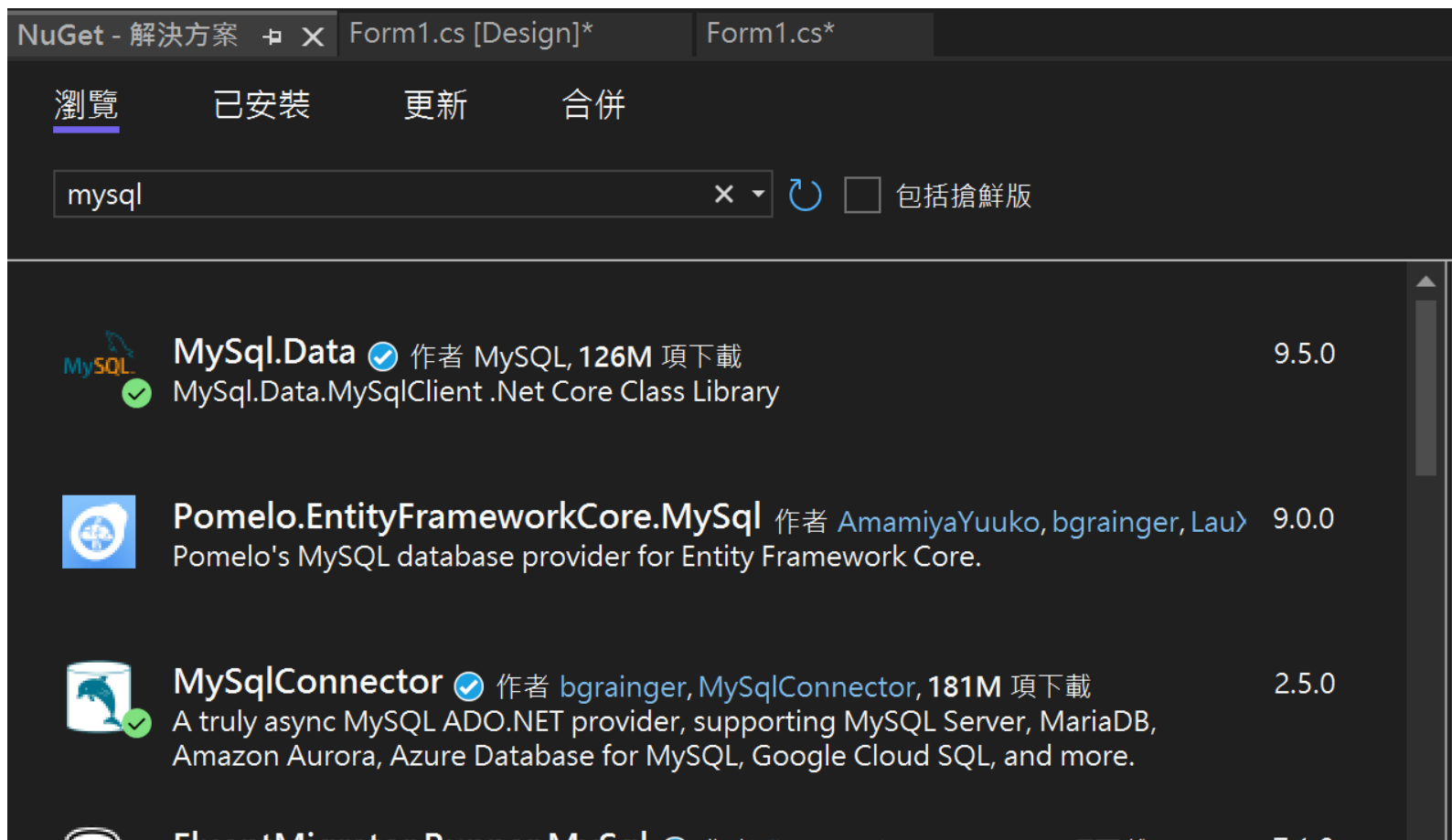


透過HeidiSQL調整使用者權限

- 以root登入後



透過C#連接(1) – 安裝套件



透過C#連接(2) - 建立資料庫連線並查詢

- 引用 MySql.Data.MySqlClient

```
MySqlConnection connection;
const string database = "member";//資料庫名稱
const string databaseServer = "localhost";//資料庫伺服器
const string databaseUser = "T00001";//資料庫使用者
const string databasePassword = "123456";//資料庫密碼
private void button2_Click(object sender, EventArgs e)
{
    string connectionString =
        $"server={databaseServer};" + $"user={databaseUser};" +
        $"password={databasePassword};" + $"database={database};" +
        "charset=utf8;";
    using (connection = new MySqlConnection(connectionString))
    {
        connection.Open(); //資料庫連線
        string sql = "SELECT * FROM user";
        using (MySqlCommand cmd = new MySqlCommand(sql, connection))
        {
            using (MySqlDataReader reader = cmd.ExecuteReader())
            {
                dataGridView1.Rows.Clear();
                while (reader.Read())
                {
                    int id = reader.GetInt32("id");
                    string username = reader.GetString("name");
                    dataGridView1.Rows.Add(id.ToString(), username);
                }
            }
        }
        connection.Close(); //資料庫斷線
    }
}
```

透過C#連接(3) – 新增資料

```
private void button5_Click(object sender, EventArgs e)
{
    string connectionString =
        $"server={databaseServer};" + $"user={databaseUser};" +
        $"password={databasePassword};" + $"database={database};" +
        "charset=utf8;";
    using (connection = new MySqlConnection(connectionString))
    {
        connection.Open();

        string sql =
            "INSERT INTO user (id, name) VALUES (@id, @username)";
        using (MySqlCommand insertCommand = new MySqlCommand(sql, connection))
        {
            insertCommand.Parameters.AddWithValue("@id", textBox1.Text);
            insertCommand.Parameters.AddWithValue("@username", textBox2.Text);

            int rowsAffected = insertCommand.ExecuteNonQuery();

        }
    }
}
```

透過C#連接(4) - 更新資料

```
private void button3_Click_1(object sender, EventArgs e)
{
    string connectionString =
        $"server={databaseServer};" + $"user={databaseUser};" +
        $"password={databasePassword};" + $"database={database};" +
        "charset=utf8;";
    using (MySqlConnection connection = new MySqlConnection(connectionString))
    {
        connection.Open();

        string sql = "UPDATE user SET name = @name WHERE id = @id";
        using (MySqlCommand updateCommand = new MySqlCommand(sql, connection))
        {
            updateCommand.Parameters.AddWithValue("@name", textBox2.Text);
            updateCommand.Parameters.AddWithValue("@id", textBox1.Text);

            int rowsAffected = updateCommand.ExecuteNonQuery();
        }
    }
}
```


透過C#連接(5) – 刪除資料

```
private void button6_Click(object sender, EventArgs e)
{
    string connectionString =
        $"server={databaseServer};" + $"user={databaseUser};" +
        $"password={databasePassword};" + $"database={database};" +
        "charset=utf8;";
    using (connection = new MySqlConnection(connectionString))
    {
        connection.Open();
        string sql = "DELETE FROM user WHERE id = @id";
        using (MySqlCommand deleteCommand = new MySqlCommand(sql, connection))
        {
            deleteCommand.Parameters.AddWithValue("@id", textBox1.Text);
            int rowsAffected = deleteCommand.ExecuteNonQuery();

        }
        connection.Close(); //資料庫斷線
    }
}
```